Agent Postmortem Generator

Part 9

Module: Web Interface (UI)

Local browser-based presentation layer for inspecting completed runs, generated postmortems, and supporting evidence without reinterpreting the source analysis.

Project	Agent Postmortem Generator
Module	Web Interface (UI)
Purpose	Local browser-based presentation layer for inspecting completed runs, generated postmortems, and supporting evidence without reinterpreting the source analysis.

1. Executive Overview

The Web Interface is the human-facing presentation layer of the system. It runs locally in the user's browser, connects to a local run session bundle or local runtime endpoint, and renders the outputs produced by the deterministic pipeline. Its job is to make a completed investigation easy to inspect without changing the underlying analysis or introducing any new interpretation layer.

This module exists because the CLI is efficient for automation, but people still need to read, compare, and navigate evidence. The UI provides structure around that experience: a run summary, a generated postmortem view, linked evidence, execution graph visualization, and raw artifact access. It does not parse raw traces, normalize data, infer causality, or rewrite conclusions.

In the system stack, the UI sits after the Output Layer. It consumes artifacts already produced by downstream modules and presents them in a stable, local-only browser experience.

```
Input -> Parser -> Validation -> Normalizer -> Canonical Trace Model
  -> Analysis Engine -> Postmortem Generator -> Output Layer -> Web Interface (UI)
```

2. Problem Statement

Failed AI agent runs are difficult to inspect because the useful evidence is distributed across timeline events, tool calls, intermediate thoughts, errors, and generated report artifacts. Humans can read the raw log stream, but the cognitive load is high and the signal is buried.

This problem is hard for three reasons. First, the data is nested and the relationships are not obvious. Second, the same run may produce multiple artifact types that need to remain synchronized. Third, a browser UI can accidentally become a second analysis engine if it starts inferring relationships, laying out graphs nondeterministically, or mutating content for convenience.

The MVP needs this module because a usable tool has to support human inspection. Without a web interface, the project is functional but awkward. With it, a user can move from summary to evidence to raw artifact without losing context or trusting a probabilistic layer to explain the result.

3. Goals

- Present a completed run as a coherent investigation workspace, not a pile of files.
- Make the generated postmortem and supporting evidence easy to compare side by side.
- Keep all rendering deterministic, local, and read-only.
- Handle missing or partial artifacts gracefully without inventing replacements.
- Provide a low-friction path from run selection to report inspection.
- Support keyboard navigation and readable, accessible defaults.

Success means the UI helps an engineer answer the question, "what failed, where did it fail first, and what evidence supports that conclusion?" without leaving the local environment or relying on hidden browser logic.

4. Non-Goals

- Raw trace ingestion, parsing, validation, normalization, or analysis.
- Authentication, user accounts, collaboration, or remote sharing.
- Cloud storage, sync, or any server-side multi-user workspace.
- In-browser editing of source artifacts or report conclusions.
- Custom dashboards, analytics, trend charts, or workspace management for many runs.
- AI-generated UI text, free-form explanations, or probabilistic summarization.
- Rich graph layout generation in the browser if that requires heuristic placement.
- Persistent database state beyond small local UI preferences.

Later versions may add cross-run comparison, a workspace library, plugin-based panels, or PDF export launchers. Those are natural extensions, but they are not required to prove the core value proposition.

5. Responsibilities

Owns	Must Never Own
Shell, routing, and local page composition	Trace parsing or normalization
Loading a completed run session bundle	Causal inference or failure attribution
Rendering report, graph, evidence, and raw artifact views	Changing the report content
Displaying local validation and loading errors	Guessing missing data
Managing transient UI state and local preferences	Source-of-truth persistence
Sanitizing and safely rendering content	Executing untrusted scripts from artifacts

6. Inputs

Expected inputs

- A local run session manifest, either loaded from the local runtime or selected from disk.
- Artifact descriptors that point to generated Markdown, JSON, SVG, Mermaid source, logs, or graph data.
- Route state such as session ID, selected artifact, selected evidence item, and optional search state.
- Small local preferences such as theme, density, and last opened run.

Validation rules

- The manifest must declare a schema version and a stable session identifier.
- Every referenced artifact must exist, be readable, and have a declared type or render hint.
- Artifact ordering must follow the manifest, not discovery timing or browser fetch timing.
- Markdown and HTML content must be sanitized before rendering.
- Graph data must either include stable positions or be explicitly marked as no-layout-preview.
- Unknown fields should be preserved in the manifest object but ignored by the UI unless explicitly supported.

Assumptions

- The UI runs locally and has no dependency on remote services.

- The output layer has already produced the authoritative artifacts.
- The graph module, if used, has already decided node and edge identity.
- The browser may be offline or disconnected from the network at all times.

Malformed input handling

- Missing manifest: show a local error page with the exact expected location or route.
- Missing artifact: keep the session shell open and show a per-panel error state.
- Invalid markdown or HTML: render a safe fallback view and strip unsafe markup.
- Invalid graph data: show the raw source text or a structured placeholder, not a guessed diagram.
- Unsupported artifact type: display metadata and a plain-text preview if possible.

7. Outputs

The UI does not create authoritative project artifacts. Its outputs are presentation-only surfaces and transient local UI state.

Output surface	Description	Persistence
Run overview	One-page summary of the selected session and its state	Ephemeral browser view
Report viewer	Rendered postmortem content with anchors and sections	Ephemeral browser view
Graph viewer	Precomputed execution graph or SVG preview	Ephemeral browser view
Evidence panel	Linked evidence item details and source references	Ephemeral browser view
Raw artifact viewer	Source markdown, JSON, or logs for inspection	Ephemeral browser view
Local preferences	Theme, last session, and UI density	Browser storage only

Any export or download action remains a pass-through action to already generated local files. The UI does not rewrite the report, reformat the graph, or produce a separate source of truth.

8. Internal Design

Main subcomponents

Subcomponent	Responsibility
App Shell and Router	Owns page structure, route resolution, and view switching.
Session Loader	Loads and validates the local run manifest or session bundle.
Artifact Registry	Indexes available artifacts and exposes stable

	metadata for the views.
Overview Panel	Shows the run summary, status, counts, and warnings.
Report Viewer	Renders the generated postmortem with anchors and linked evidence.
Graph Viewer	Displays precomputed graph content without inventing layout.
Evidence Drawer	Shows selected evidence details and source references.
Raw Artifact Viewer	Presents markdown, JSON, or log content in safe read-only form.
Error and Empty State Layer	Handles missing data, partial bundles, and invalid inputs.
Local Preferences Store	Persists small UI preferences only.

Workflow

```
1. User opens the UI with a selected local session or recent run.
2. Session Loader validates the manifest and builds the artifact index.
3. App Shell renders the overview first so the user sees context immediately.
4. Report Viewer and Graph Viewer load lazily as the user switches tabs.
5. Evidence selections move focus between report anchors, graph nodes, and raw artifact
spans.
6. Error states stay local to the affected panel; the rest of the UI remains usable.
```

Data flow

```
Session Manifest
   -> Artifact Registry
      -> Overview Panel
      -> Report Viewer
      -> Graph Viewer
      -> Evidence Drawer
      -> Raw Artifact Viewer
      -> Local Preferences Store
```

Important internal decisions

- The UI must never recompute analysis results or failure relationships.
- Graph rendering should use precomputed graph coordinates or an explicitly deterministic renderer; it must not use browser-driven force layout for canonical views.
- Markdown rendering must be sanitized and limited to the supported subset.
- The view state must be serializable and recoverable so the user can refresh without losing context.
- Large artifacts should be loaded lazily and only when the corresponding panel is active.
- All visible ordering should come from the manifest or explicit identifiers, never fetch timing.

9. Data Model

Entity	Key fields	Why it exists
UI Session	session_id, source_kind, status, created_at, run_name, counts, warnings, artifact_refs	Represents one local investigation workspace.
Artifact Descriptor	artifact_id, type, label, mime_type, href/path, size_bytes, checksum, render_mode	Provides stable metadata for loading and rendering.
Evidence Reference	evidence_id, title, source_artifact, span or node reference, linked_artifacts	Connects report statements back to source evidence.
View State	active_tab, active_artifact_id, active_evidence_id, search_query, scroll_anchor	Keeps navigation predictable and refreshable.
User Preference	theme, density, last_opened_session	Stores harmless local convenience settings.

Relationships

- A UI Session owns many Artifact Descriptors.
- An Evidence Reference points to one primary artifact and may link to others.
- View State references a Session and a selected subset of artifacts or evidence items.
- User Preference is independent of any one session and can survive refreshes.

This model exists to keep the UI honest. The browser does not need the entire canonical trace model to function; it needs a narrow session contract that is stable, inspectable, and easy to test.

10. Algorithms / Processing Logic

Step-by-step processing

1. Resolve the selected local session source from route state, recent history, or an explicit open action.
2. Load the session manifest and validate the minimum required fields.
3. Build a stable artifact registry from the manifest and preserve manifest ordering.
4. Render the overview and any lightweight metadata before heavy panels.
5. Load the active panel content on demand and sanitize it before display.
6. Resolve evidence links by manifest ID and open the associated panel or anchor.
7. Persist only transient UI preferences and the current session pointer.

Edge cases

- If the graph view lacks coordinates, render a structured fallback and keep the session open.
- If a report section has no evidence anchors, display the text without synthesizing links.
- If markdown contains unsupported HTML, strip it instead of trying to interpret it.
- If the session bundle is incomplete, mark missing panels clearly and keep available panels functional.

- If content is extremely large, render it with a controlled preview and explicit loading boundaries rather than freezing the whole interface.

Decision rules

- Prefer manifest order over alphabetical order unless the manifest is silent.
- Prefer explicit evidence IDs over text matching.
- Prefer safe fallback over best-guess rendering.
- Prefer stable IDs and anchors over DOM-inferred relationships.
- Prefer no graph over a guessed graph when layout data is absent.

Deterministic behavior requirements

- The same session manifest must produce the same visible ordering and default selection.
- The same artifact set must map to the same anchors and link targets.
- The UI must not depend on network timing, random IDs, or browser-specific layout heuristics for core views.
- Any client-side rendering library used for markdown or graph previews must be version-locked and treated as deterministic infrastructure, not a creative engine.

11. Tradeoffs

Tradeoff	Chosen approach	Reason
Simplicity vs flexibility	A small read-only local UI instead of a generalized platform	The MVP needs inspection, not a feature forest.
Generality vs specificity	Optimized for session bundles and generated artifacts	General-purpose file browsing would dilute the core use case.
Determinism vs heuristic inference	Strict manifest-driven rendering	Hidden browser logic is bad engineering and worse debugging.
Local-only vs remote collaboration	Local-only	The product promise is privacy, predictability, and laptop-local execution.

The preferred design is intentionally constrained. It is less clever than a dashboard framework, but it is far easier to reason about, test, and maintain. That is a good trade when the real product value comes from accurate evidence review, not from flashy UI mechanics.

12. Risks and Failure Modes

- Browser rendering libraries can introduce subtle differences across environments.
- A rich markdown renderer can become an XSS risk if sanitization is treated casually.
- Graph visualization can drift into nondeterminism if layout is left to the client.
- Large traces can strain browser memory or slow the main thread.
- Session manifests can go stale if files are moved or deleted after generation.
- The UI can accidentally duplicate backend logic if ownership boundaries are not enforced.
- Path handling across Windows, macOS, and Linux can expose platform-specific edge cases.

The most important maintenance risk is scope creep. Once the UI starts deciding what the data means, the architecture stops being modular. The session contract must remain the line in the sand.

13. Testing Strategy

Test type	What it protects	Example cases
Unit tests	Manifest parsing, state transitions, sanitization, link resolution	Missing artifact, invalid IDs, tab selection, preference persistence
Integration tests	Local session loading and multi-panel rendering	Overview to report to graph navigation, partial bundle recovery
Fixture-driven tests	Stable coverage for real-world bundle shapes	Minimal session, large session, missing graph, missing markdown
Regression tests	Visual and behavior stability	Snapshot diffs for core panes, ordering stability, keyboard navigation
Accessibility tests	Usable keyboard and screen-reader flows	Focus order, contrast, labels, skip navigation

- Use deterministic fixtures for the same manifest and artifact set.
- Add screenshot-based regression checks for the shell, report viewer, graph viewer, and error states.
- Verify that all cross-links still work after refactors.
- Test sanitization on markdown and any user-provided or trace-derived text.
- Confirm that no test depends on network access.

14. Implementation Checklist

8. Define the local session contract and schema version for the UI.
9. Implement the shell, routing, and session loader first.
10. Build the artifact registry and stable navigation model.
11. Add the overview panel and the read-only report viewer.
12. Add evidence navigation and a raw artifact viewer.
13. Add graph rendering that consumes precomputed graph data only.
14. Add error states for missing or malformed artifacts.
15. Persist only harmless local preferences.
16. Create deterministic fixtures for minimal, partial, and large sessions.
17. Run accessibility, snapshot, and cross-browser checks before release.

The implementation order matters because the shell and contract are the foundation. If the data model is shaky, everything above it becomes decorative trouble.

15. Future Expansion

- Cross-run comparison mode for two completed sessions.

- Workspace library that groups multiple local sessions by project or folder.
- Optional PDF preview and export launch points.
- Plugin slots for custom panels or organization-specific views.
- Local full-text search over a bounded set of saved sessions.
- Annotation support that remains local and does not change canonical artifacts.

These are reasonable follow-ons after the MVP has proved that a clean local inspection experience is useful. None of them are required to justify the first release.

16. Final Recommendation

The design is MVP-ready if the module stays read-only, local-only, and manifest-driven. That keeps the UI honest, the implementation compact, and the testing surface manageable for a small team.

The main simplification still recommended is to avoid any browser-side graph layout generation for canonical views. Precompute layout upstream or display a deterministic fallback. That one choice removes a surprising amount of future pain, which is almost rude of reality to do but there it is.